Implementation and reproduction guide

Review edition, 15 November 2026.

1. Obtain the reviewed branch

Use the repository's feature branch and record the commit before running. PR #1 remains the review location. Do not substitute `main`, which may not contain the simulator until a reviewed merge.

```
git clone https://github.com/sunnatbek06/FreeRTOSproject.git
cd FreeRTOSproject
git checkout feature/spacecraft-simulator
git rev-parse HEAD
```

2. Linux build

The validated platform is Ubuntu 22.04. CMake requires 3.20 or later. The first configuration downloads FreeRTOS-Kernel V11.1.0. For offline builds provide an existing kernel tree with `-DFREERTOS_KERNEL_PATH=/absolute/path`.

```
sudo apt-get update
sudo apt-get install -y build-essential cmake ninja-build git python3
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
python3 -m unittest discover -s tests -v
python3 scripts/check_cli.py build/spacecraft_sim
```

On macOS, edit and analyze locally, then execute in an Ubuntu VM/container or GitHub Actions. No Linux runtime was present on the development Mac during this review. Native simulator execution on that Mac was not verified. The repository's Apple CMake guard is an explicit project restriction. Upstream POSIX source contains Apple conditionals, so blanket incompatibility is not asserted.

3. Smoke test and validation

```
mkdir -p results/smoke
./build/spacecraft_sim --duration 9 --policy dynamic --run-id smoke01 > results/smoke/events.csv 2> re
sults/smoke/diagnostics.txt
python3 scripts/validate_evidence.py results/smoke/events.csv results/smoke/diagnostics.txt --duration
 9
python3 scripts/analyze.py results/smoke/events.csv results/smoke/summary.csv
```

CSV is stdout; diagnostics are stderr. Do not merge these streams. A normal 9-second mission includes additional shutdown time. The validator must pass before interpreting latency summaries. A deadline miss is a measured outcome, not a failed consistency check.

4. Original full experiment suite

```
python3 scripts/run_matrix.py --executable build/spacecraft_sim --duration 60 --repetitions 3 --output
 results/full
```

This executes E1-E9 three times sequentially, about 29-32 minutes including shutdown. Each run preserves CSV, stderr and a manifest. The CI revision uses three isolated repetition jobs, each executing all nine scenarios sequentially on one runner. Distinct repetitions may use different virtual machines; they are replications across runners rather than three repetitions on an invariant physical host. The earlier November 13 suite ran all 27 sequentially in one job.

5. Supplementary CPU contention suite

```
python3 scripts/run_contention.py --executable build/spacecraft_sim --repetitions 3 --output results/contention
```

Six scenarios run for 15 seconds each, three repetitions, on one runner in repetition-major order. CPU demand is 40,000 us for C1-C5 and 20,000 us for C6. Telemetry doubles this demand during orbit insertion. Emergency interval is 100 ms. C1-C4 compare policies. C5 uses telemetry capacity 2. C6 reduces CPU demand. These additions expose competition that the original loop-only workload largely hides.

A single case can be reproduced with:

```
./build/spacecraft_sim --duration 15 --policy dynamic --service-us 40000 --emergency-interval 100 --run-id contention01 > events.csv 2> diagnostics.txt
python3 scripts/validate_evidence.py events.csv diagnostics.txt --duration 15
```

6. Reading output

Each event has run ID, relative kernel tick, phase, channel, event, depth, latency and deadline fields. Phase-control rows reuse latency/deadline fields for worker priorities. Only completed, handled and deadline_miss events belong in latency statistics. A zero-tick sample is below the observation resolution, not zero physical time.

COUNTS `sent_*` means accepted into a queue, not all source attempts. Attempt count can be reconstructed only as accepted plus dropped. `processed_*` includes deadline misses. `dropped_log` must be zero. Remaining queue depths must all be zero. Do not average per-run p95 values and label the result a pooled p95; the report recomputes pooled values from raw samples and also reports run-level variation.

7. Demonstration sequence

Build and show the test result. Run the 9-second smoke case. Show the two phase transition rows and three priority updates in CSV. Show reconciled COUNTS. Run a 15-second CPU-demand case. Explain the difference between successfully processing a message and meeting its deadline. End with the report's limitations: source release delay, host jitter, coarse timing and absence of radio/hardware validation.

8. Troubleshooting

A missing compiler/CMake prevents build; install the packages above. A FetchContent download failure needs network access or a local kernel path. A failed trace validator needs inspection of CSV and stderr, not manual deletion of failing rows. Queue drops may be legitimate overload outcomes. Logger loss or accepted/completed mismatch invalidates complete latency analysis. A final artifact link can expire, so retain the local evidence archives and SHA256 manifest in this package.